Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский государственный электротехнический университет «ЛЭТИ» им. В.И. Ульянова (Ленина)»
кафедра ВТ

ОТЧЕТ

по лабораторной работе №4

по дисциплине «Технологии Web-программирования»

Тема: «ИНТЕРНАЦИОНАЛИЗАЦИЯ web-ПРИЛОЖЕНИЙ»

Студенты гр. 4314 _____ Селикаев Р. К.

Преподаватель _____ Павловский М. Г.

Санкт-Петербург
2026

Цель работы: знакомство со способами отображения данных на различных языках при использовании файлов ресурсов.

Порядок выполнения работы:

1. Проанализировать разработанное в лабораторной работе № 3 приложение и определить, для каких элементов пользовательского интерфейса можно провести локализацию.
2. Создать новый Maven web-проект, аналогичный лабораторной работе № 3, и добавить в него каталог ресурсов.
3. Создать файлы ресурсов для английского и русского языков и включить их в каталог ресурсов.
4. Внести изменения в сервлет приложения для возможности переключения интерфейса на английский и русский языки.
5. Запустить приложения в двух режимах и снять скриншоты интерфейсов на английском и русском языках.
6. Сгенерировать документацию проекта с помощью команды Maven build и просмотреть ее в браузере.

Создание и загрузка файлов ресурсов

Для хранения данных (ресурсов), зависящих от локали, используют класс *ResourceBundle*, который хранится в библиотеке *java.util.ResourceBundle*. Этот класс абстрактный, поэтому получить его экземпляр можно исключительно с помощью статического метода *getBundle*, имеющего несколько перегруженных реализаций. В данной лабораторной работе будет использован вариант, в котором для хранения ресурсов служит файл с расширением properties.

Ресурсный файл создается как обычный текстовый файл, содержащий перечень строк вида «ключ=значение». Ключ – это имя, которое может быть использовано программой для получения значения. Как в ключах, так и в значениях допускаются пробелы. Текст в файле должен быть записан в кодировке Unicode. Например, файлы ресурсов с русским и английским переводами компонентов интерфейса приложения, отображающего список комиксов читателя, будут содержать следующую информацию (см. рис. 1, 2):

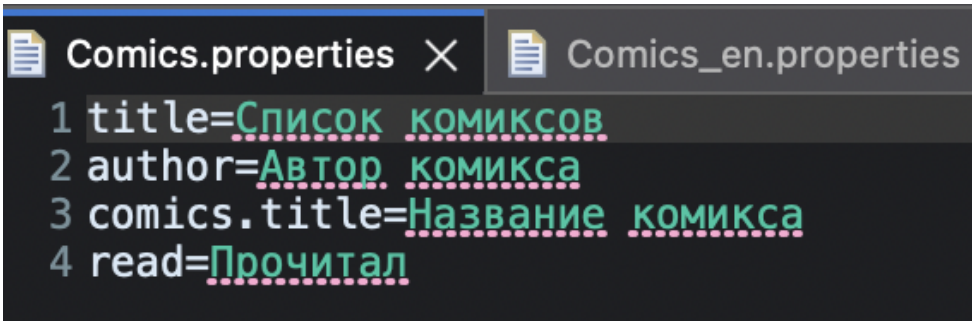


Рис. 1 – *Распечатка файла ресурсов Comics.properties*

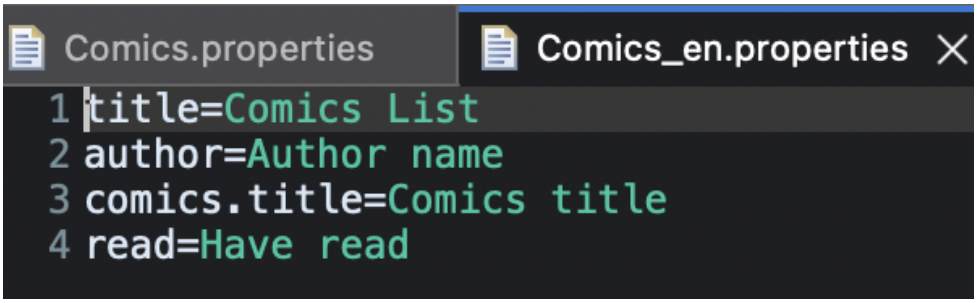


Рис. 2 – *Распечатка файла ресурсов Comics_en.properties*

Локализация компонентов интерфейса и кода

Для локализации компонентов интерфейса и строк сообщений, появляющихся в коде приложения, необходимо заменить жестко запрограммированные названия и описания выражениями, которые ссылаются на переведенные названия и строки в локализованном файле ресурсов. Если необходимо переключение локализаций, то целесообразно информацию о нужной локализации передавать в качестве параметра сервлету или хранить эту информацию в сессии, так как она требуется всем страницам приложения в рамках сессии.

Ниже приведена распечатка кода сервлета *LocaleServlet.java*, который по запросу клиента выводит список книг читателя на русском или английском языке. Язык задается параметром *lang* в адресе страницы, который может принимать значения «*ru*» или «*en*».

LocaleServlet.java

```
package lab.web;

import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Locale;
import java.util.ResourceBundle;

/**
 * Servlet implementation class LocaleServlet.
 */
public class LocaleServlet extends HttpServlet {
   private static final long serialVersionUID = 1L;

   public LocaleServlet() {
      super();
   }
   protected void processRequest(HttpServletRequest request, HttpServletResponse
response)
         throws ServletException, IOException {

      // Чтение параметра lang из запроса
      String lang = request.getParameter("lang");
      if (lang == null) {
         response.sendError(HttpServletResponse.SC_NOT_ACCEPTABLE,
               "Ожидался параметр lang");
         return;
      }
      if (!"en".equalsIgnoreCase(lang) && !"ru".equalsIgnoreCase(lang)) {
         response.sendError(HttpServletResponse.SC_NOT_ACCEPTABLE,
               "Параметр lang может принимать значение en или ru");
         return;
      }

      // Задание локали
      Locale locale = "en".equalsIgnoreCase(lang)
            ? Locale.ENGLISH
            : new Locale("ru", "RU");

      // Загрузка ресурсов
```

```
        ResourceBundle res = ResourceBundle.getBundle("Comics", locale);

        // Задание типа содержимого
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();

        try {
            out.println("<html>");
            out.println("<head><title>" + res.getString("title") + "</title></head>");
            out.println("<body>");
            out.println("<h1>" + res.getString("title") + "</h1>");
            out.println("<table border='1'>");
            out.println("<tr><td><b>" + res.getString("author") + "</b></td>"
                    + "<td><b>" + res.getString("comics.title") + "</b></td>"
                    + "<td><b>" + res.getString("read") + "</b></td></tr>");
            out.println("<tr><td>Джонатан Хикман</td><td>Секретные
Войны</td><td>Да</td></tr>");
            out.println("<tr><td>Брайан Бендис</td><td>День
M</td><td>Да</td></tr>");
            out.println("<tr><td>Донни
Кейтс</td><td>Веном</td><td>Нет</td></tr>");
            out.println("</table>");
            out.println("</body>");
            out.println("</html>");
        } finally {
            out.close();
        }
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
}
```

Результаты запуска web-приложения

После редактирования кода сервлета для корректной локализации языков, мы запускаем наш проект через *Maven install* по аналогии с предыдущими лабораторными работами и импортируем получившийся war-файл проекта на наш web-сервер *TomCat*. Вслед за этим мы запускаем наш сервер и открываем две ссылки в браузере с разным параметров *lang* и смотрим на получившийся результат тестовой интернационализации web-приложения. Результаты работы можно увидеть на рисунках 3 и 4.

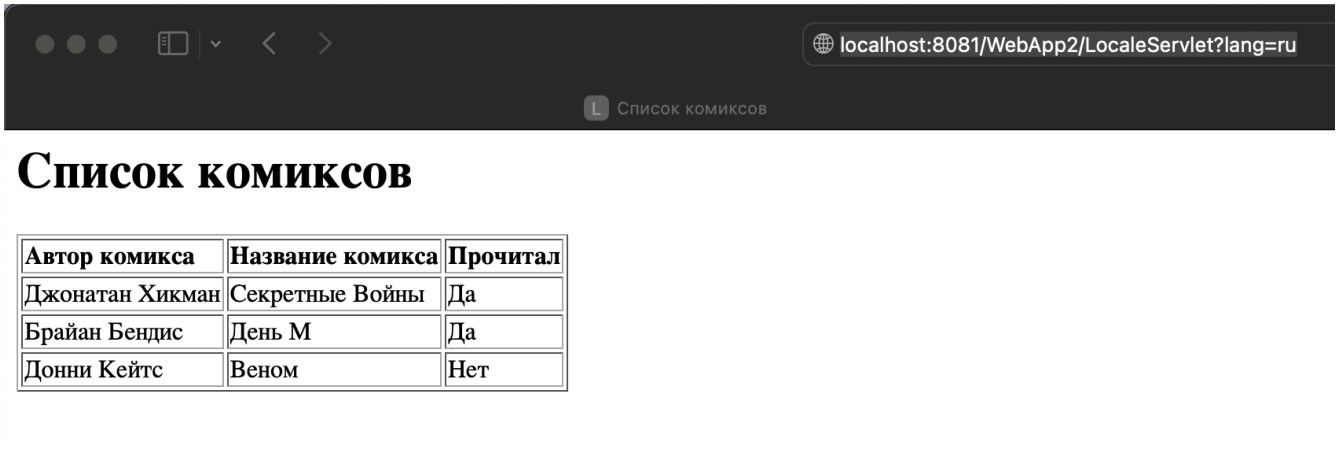


Рис. 3 – *Web-приложение с русским интерфейсом*

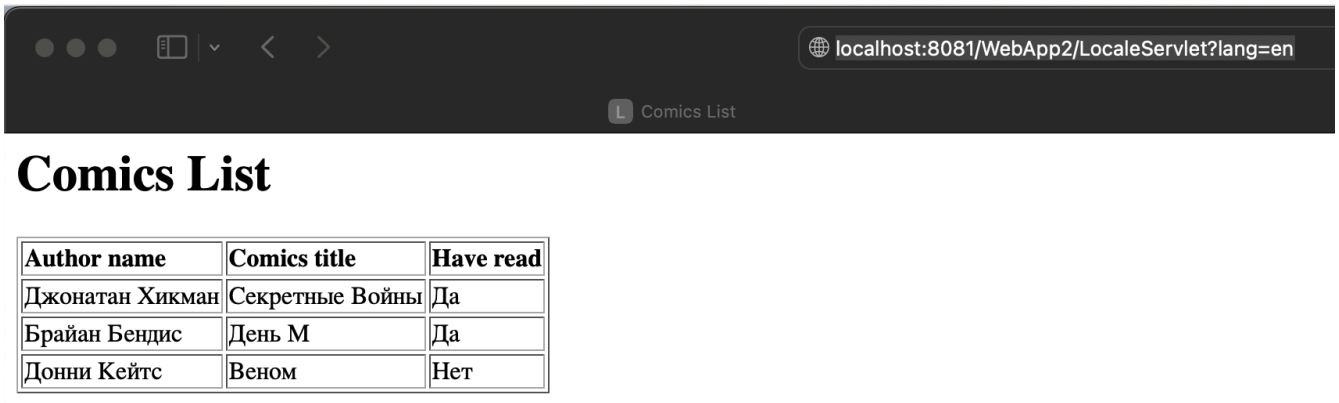


Рис. 4 – *Web-приложение с английским интерфейсом*

Документация проекта

Для документации проекта использовалась команда терминала *mvn javadoc:javadoc* в среде нашего Maven проекта. В итоге мы получили файл документации в директории *target/reports/apidocs/lab/web*. Содержание файла документации отображено на рисунке 5.



Class LocaleServlet

java.lang.Object
jakarta.servlet.GenericServlet
jakarta.servlet.http.HttpServlet
lab.web.LocaleServlet

All Implemented Interfaces:
`jakarta.servlet.Servlet, jakarta.servlet.ServletConfig, Serializable`

---

public class LocaleServlet extends jakarta.servlet.http.HttpServlet
Servlet implementation class LocaleServlet.

See Also:
- Serialized Form

Constructor Summary

Constructors
Constructor
Description
`LocaleServlet()`

Method Summary

All Methods | Instance Methods | Concrete Methods
Modifier and Type
Method
Description
`protected void doGet(jakarta.servlet.http.HttpServletRequest request, jakarta.servlet.http.HttpServletResponse response)`

`protected void doPost(jakarta.servlet.http.HttpServletRequest request, jakarta.servlet.http.HttpServletResponse response)`

`protected void processRequest(jakarta.servlet.http.HttpServletRequest request, jakarta.servlet.http.HttpServletResponse response)`

Methods inherited from class jakarta.servlet.http.HttpServlet

`doDelete, doHead, doOptions, doPut, doTrace, getLastModified, service, service`

Methods inherited from class jakarta.servlet.GenericServlet

`destroy, getInitParameter, getInitParameterNames, getServletConfig, getServletContext, getServletInfo, getServletName, init, init, log, log`

Methods inherited from class java.lang.Object

`clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait`

Constructor Details

LocaleServlet

public LocaleServlet()

Method Details

processRequest

protected void processRequest(jakarta.servlet.http.HttpServletRequest request, jakarta.servlet.http.HttpServletResponse response) throws jakarta.servlet.ServletException, IOException
Parameters:
`request` - HTTP-запрос
`response` - HTTP-ответ
Throws:
`jakarta.servlet.ServletException` - при ошибке сервлета
`IOException` - при ошибке ввода-вывода

doGet

protected void doGet(jakarta.servlet.http.HttpServletRequest request, jakarta.servlet.http.HttpServletResponse response) throws jakarta.servlet.ServletException, IOException
Overrides:
`doGet` in class `jakarta.servlet.http.HttpServlet`
Throws:
`jakarta.servlet.ServletException`
`IOException`

doPost

protected void doPost(jakarta.servlet.http.HttpServletRequest request, jakarta.servlet.http.HttpServletResponse response) throws jakarta.servlet.ServletException, IOException
Overrides:
`doPost` in class `jakarta.servlet.http.HttpServlet`
Throws:
`jakarta.servlet.ServletException`
`IOException`

Рис. 5 – *Документация проекта лабораторной работы*

Вывод по лабораторной работе: в результате выполнения лабораторной работы была реализована интернационализация web-приложения с использованием файлов ресурсов. Мы научились локализовывать интерфейс, а также подключать языковые пакеты.

Приложение

Ссылка на репозиторий с проектом лабораторной работы представлена ниже:
https://github.com/romaselikaev/web-tech/tree/main/lab4